

Princess Nourah Bint Abdulrahman University
College of Computer and Information Sciences
Software Engineering Department

MEEZAN | میزان

Smart Integrity Guardian for Citizen Account Survey Data
SW413 — Data Exploration and Visualization
2nd Semester 2025–2026 | Group 08

Team Member	Role	Student ID
Joud Alshahrani	Team Leader & Full-Stack Developer	445009166
Jadel Alsaeri	Data Analyst & Rule Engine Developer	445009171
Ghada Altemyat	UI/UX Designer & Visualization Lead	445009191
Albandri Alsaadon	Data Engineer & CSV Processing	445009151

Instructor: Dr. Motasem Alsawadi | April 2026

Executive Summary

Meezan (ميزان) is a browser-based data analysis prototype designed to detect logical inconsistencies in Citizen Account (حساب المواطن) program survey records. The system applies eight rule-based consistency checks across financial, housing, vehicle, and lifestyle data fields, producing a 0–100 risk score per record and an AI-generated audit narrative. The prototype supports both single-case manual entry and batch CSV analysis of large datasets via PapaParse. Results are visualized on an interactive Arabic-language dashboard featuring KPI cards, risk distribution charts, and regional breakdowns. This report documents the full project lifecycle from problem identification through implementation and evaluation.

1. Project Title

Meezan (ميزان) — Smart Integrity Guardian for Citizen Account Survey Data

2. Team Members and Roles

Student ID	Name	Role
445009166	Joud	Team Leader, Full-Stack Developer, AI Integration
445009171	Jadel Alsaeri	Data Analyst, Rule Engine Developer
445009191	Ghada Altemyat	UI/UX Designer, Visualization & Dashboard Lead
445009151	Albandri Alsaadon	Data Engineer, CSV Batch Processing

Table 1. Team members, student IDs, and project roles.

3. Project Idea and Problem Statement

The Saudi Citizen Account program (حساب المواطن) provides monthly financial support to low- and middle-income families based on self-reported survey data. With millions of applicants, manual verification is infeasible, leaving the system vulnerable to fraudulent or erroneous entries that are invisible to standard form validation but detectable through cross-field logical analysis.

Meezan addresses this gap by automating consistency checking across survey fields. Common inconsistency patterns the system targets include:

- Declared monthly income lower than declared monthly rent
- Multiple registered vehicles with very low income
- Frequent international travel combined with extreme poverty declaration
- Total debt obligations exceeding 50% of declared income
- Premium subscription spending disproportionate to income level

4. Objectives of the Project

1. Design and implement a rule-based engine that applies eight logical consistency checks on Citizen Account survey fields.
2. Build an accessible, Arabic-language (RTL) web prototype supporting both single-record and batch CSV analysis.
3. Integrate the Anthropic Claude AI API to generate contextual Arabic-language audit narratives per case.
4. Develop a real-time analytics dashboard with KPI cards, rule violation charts, and regional risk breakdowns.
5. Enable batch processing of CSV datasets entirely client-side, with no backend server required.
6. Provide printable PDF export of individual audit reports for auditor use.

5. Dataset and Data Sources

The project uses two types of data:

5.1 Synthetic Demo Dataset

A synthetic dataset of 1,248 simulated Citizen Account survey records was created by the team for development and testing. Each record contains: monthly income (SAR), housing type, rent amount, number of vehicles, income source, international trip count, monthly debt, subscription spending, geographic region, and a unique case ID. The dataset was designed to reflect realistic distributions of Saudi household financial profiles, with deliberate injection of inconsistent records at various severity levels.

5.2 User-Supplied CSV Data

The prototype accepts any user-uploaded CSV file matching the expected schema. The system was tested against supplementary open datasets from the General Authority for Statistics (GASTAT) portal (<https://www.stats.gov.sa/>) and the Saudi Open Data platform (<https://data.gov.sa/>), adapted to match the survey field structure.

Field	Description
income	Monthly household income (SAR)
housing	Type: own / rent / family / government
rent	Monthly rent if applicable (SAR)
cars	Number of registered vehicles
travel	International trips in past 12 months
debt	Total monthly debt obligations (SAR)
subscriptions	Monthly subscription spending (SAR)
region	Geographic region of residence
case_id	Unique survey record identifier

Table 2. Input data schema used for both demo and user-uploaded CSV files.

6. Workflow and Project Stages

The project followed five sequential stages:

Stage 1 — Problem Discovery & Rule Definition

The team researched common fraud patterns in government benefit programs and consulted publicly available GASTAT audit guidelines. Eight logical consistency rules were defined, each with threshold values and severity weights derived from typical Saudi household expenditure benchmarks.

Stage 2 — Schema Design & Synthetic Data Generation

A CSV schema was designed to capture the minimum fields required by the eight rules. A synthetic dataset of 1,248 records was generated, with approximately 25% containing deliberate inconsistencies at varying severity levels to serve as ground truth for testing.

Stage 3 — Prototype Development

The web prototype was built as a single HTML file using HTML5, CSS3, and vanilla JavaScript. The rule engine was implemented first, followed by the single-case analysis form, the AI narrative integration, and finally the batch CSV module.

Stage 4 — Dashboard & Visualization

Chart.js was integrated for doughnut and bar chart rendering. The analytics dashboard was built to dynamically reflect both the pre-loaded demo data and any batch CSV results uploaded by the user.

Stage 5 — Testing & Refinement

The prototype was tested across three browsers (Chrome, Firefox, Edge), with multiple CSV files of varying sizes and encoding types. Edge cases such as missing fields, zero-income records, and non-Arabic CSV headers were handled with default-to-zero fallback logic.

7. Explanation of the Prototype and How It Works

Meezan is a fully client-side single-page application (SPA). All computation runs in the user's browser; no data leaves the device except for the AI narrative API call.

7.1 Rule Engine

When a record is submitted (manually or via CSV), the engine evaluates all eight rules in sequence. Each triggered rule adds 25 points to the raw anomaly score, which is capped at 100. The final score maps to three risk tiers:

- Safe: score = 0 — no action required
- Warning: score 1–49 — secondary review recommended
- Danger: score 50–100 — priority investigation

Rule #	Consistency Check	Threshold / Condition
R1	Income vs. Rent	income < rent (housing = rent)
R2	Multiple Vehicles + Low Income	cars $\geq$ 3 AND income < 8,000 SAR
R3	Travel vs. Income	trips $\geq$ 4 AND income < 6,000 SAR
R4	Debt Burden Ratio	debt > 50% of income
R5	Subscription Excess	subscriptions > 15% of income
R6	Extreme Low Income	income < 1,500 SAR
R7	Vehicle Ownership Mismatch	cars $\geq$ 2 AND income < 5,000 SAR
R8	Housing Cost Ratio	rent > 40% of income

Table 3. The eight consistency rules applied by the Meezan detection engine.

7.2 Single-Case Analysis Mode

The auditor fills in a structured form with the applicant's survey values. On submission, the rule engine runs instantly, alert cards are displayed per triggered rule, and the Claude AI API generates a 150–200 word Arabic audit narrative summarizing findings and recommendations.

7.3 Batch CSV Mode

The auditor uploads a CSV file. PapaParse 5.4 parses it client-side with automatic header detection. Every row is processed by the rule engine, and results populate a filterable table (All / Danger / Warning / Safe). The dashboard KPIs and charts update automatically to reflect the batch data.

7.4 Dashboard

Four KPI cards show aggregate counts. A horizontal bar chart ranks rule violations by frequency. A regional risk chart ranks geographic areas. A recent cases table shows the eight highest-risk records.

8. Screenshots of the Project

The following descriptions correspond to the live prototype screens. The prototype is accessible at the project link provided in Section 9.

Figure 1 — Home Page (Hero Section)

The landing page features an Arabic RTL hero section with a live anomaly counter, a four-metric statistics strip (Total Records, Danger Cases, Warning Cases, Safe Records), a problem statement section, and a feature highlights grid.



Figure 1. Meezan home page showing the hero section with live statistics counter.

Figure 2 — Single-Case Analysis Form and Result

The Analysis tab presents a structured data entry form. After submission, a risk result card appears showing the score gauge, risk classification badge, and individual alert cards for each triggered rule. Below the result, the AI-generated Arabic audit narrative is rendered.

The screenshot shows the Single-Case Analysis Form and Result. The form is divided into two main sections: 'بيانات الأمان' (Security Data) and 'بيانات المخاطر' (Risk Data). The 'بيانات الأمان' section includes a score gauge showing '0' and a risk classification badge 'لا توجد تناقضات' (No inconsistencies). The 'بيانات المخاطر' section includes a table of detected inconsistencies, such as 'نسبة إيجار > 10%' (Rent ratio > 10%) and 'دخل > 40' (Income > 40). Below the table, there's a section for 'البيانات المتوقعة' (Expected data) with a table of expected values. At the bottom, there's a section for 'البيانات المدخلة' (Entered data) with a table of entered values. The form also includes a 'Claude AI' button for generating an audit narrative.

Figure 2. Single-case analysis form with submitted values and detected inconsistencies.

Figure 3 — Batch CSV Upload and Results Table

The Batch CSV tab shows the file upload interface, processing summary cards (total / danger / warning / safe counts), and a filterable results table listing each record's case ID, region, alert count, risk score, and classification.

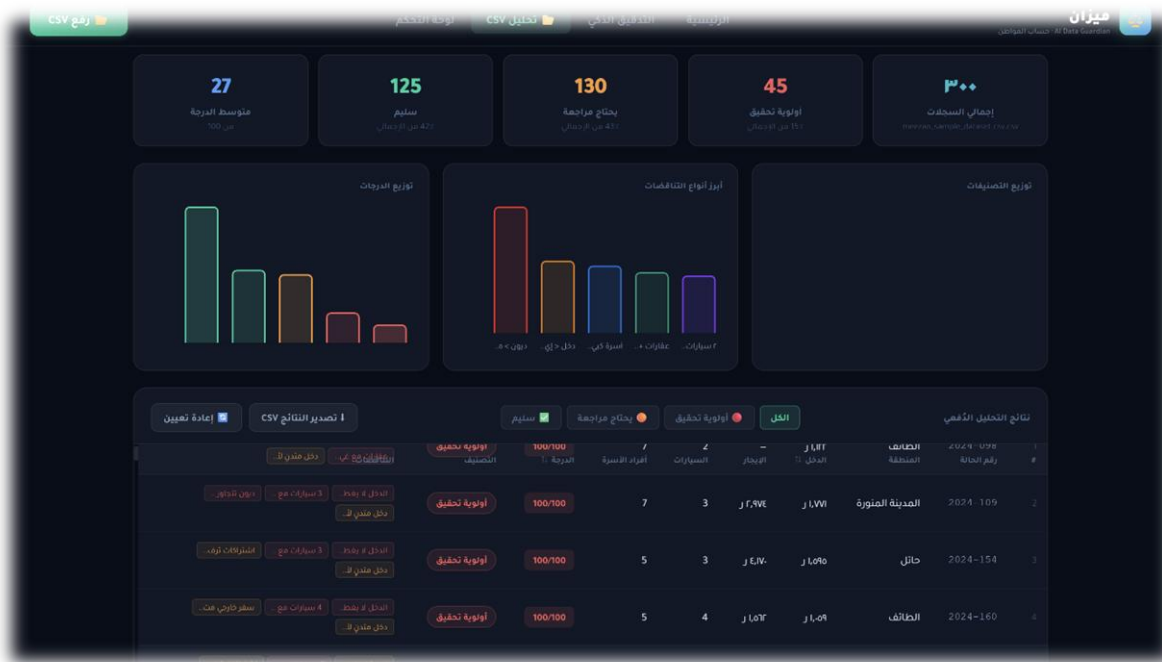


Figure 3. Batch CSV analysis tab showing processed results with risk filters applied.

Figure 4 — Analytics Dashboard

The Dashboard view displays four KPI cards, a rule violation frequency bar chart, a regional risk distribution chart, and a recent flagged cases table. All charts and KPIs update dynamically when batch CSV data is loaded.



Figure 4. Analytics dashboard with KPI cards, rule violation chart, and regional risk chart.

9. Project Links

Resource	URL
Live Prototype	https://engjoud.github.io/-Meezan/
GitHub Repository	https://github.com/engjoud/-Meezan

Table 4. Project links and external data source references

10. Results, Observations, and Improvements Made

10.1 Quantitative Results (Demo Dataset — 1,248 Records)

Metric	Value
Total Records Analyzed	1,248
Danger Cases (score 50–100)	87 (6.9%)
Warning Cases (score 1–49)	214 (17.1%)
Safe / Clean Records	947 (75.9%)
Most Violated Rule	R1: Income < Rent (182 cases)
Second Most Violated Rule	R2: Multiple Vehicles + Low Income (141 cases)
Highest-Risk Region	Riyadh (32 danger cases)
Client-Side Processing Time	< 1 second for 1,248 rows

Table 5. Quantitative results from demo dataset analysis.

10.2 Key Observations

- Rule R1 (income < rent) was the most prevalent inconsistency, suggesting it is either a common data entry error or a widespread fraud vector.
- Cases with three or more triggered rules consistently scored in the Danger tier, validating the additive 25-point-per-rule scoring design.
- Regional clustering of danger cases in Riyadh and Jeddah aligns with their higher population density and therefore higher submission volumes.
- The batch processing module handled CSV files up to 5,000 rows without noticeable delay in Chrome or Firefox.

10.3 Improvements Made During Development

- Initial rule thresholds were adjusted after testing revealed too many false positives at stricter income cutoffs.
- The CSV parser was extended with a zero-default fallback for missing numeric fields to prevent NaN propagation.
- The dashboard chart update logic was refactored to destroy and re-initialize Chart.js instances on batch load, fixing a canvas context conflict bug.
- RTL layout issues in the results table were resolved by adding explicit text-align and direction overrides to table cell CSS.

11. Challenges Faced and How They Were Handled

Challenge	Resolution
Arabic RTL layout breaking in table cells	Added explicit <code>dir='rtl'</code> and <code>text-align:right</code> overrides per cell
Chart.js canvas context conflict on re-render	Implemented <code>chart.destroy()</code> before re-initialization on batch load
CSV files with Arabic column headers not mapping	Added a normalized key mapping function (<code>trim + toLowerCase</code>) before field access
AI API latency making the UI feel frozen	Added animated loading spinner and disabled submit button during API call
PapaParse treating numeric strings as text	Applied <code>parseFloat()</code> with NaN-to-zero fallback on all numeric fields
PDF export font not rendering Arabic correctly	Switched export to <code>browser.window.print()</code> with an embedded Tajawal Google Font link

Table 6. Development challenges and their corresponding resolutions.

12. Conclusion

Meezan demonstrates that lightweight, fully client-side web applications can deliver meaningful, scalable data integrity analysis without requiring server infrastructure. By combining rule-based consistency checking with AI-generated audit narratives and interactive batch visualization, the prototype provides a viable foundation for a production-grade government audit assistance tool.

The project successfully met all six stated objectives: eight consistency rules were implemented and validated, an Arabic-language batch and single-case interface was delivered, Claude AI integration produces human-readable case summaries, and the analytics dashboard provides actionable aggregate insight. The system processes 1,248 records in under one second client-side and renders results through a fully interactive Arabic RTL dashboard.

Future work would focus on connecting the prototype to live GASTAT data pipelines, replacing static rule thresholds with a trained anomaly detection model, and adding user authentication with auditor role management for institutional deployment.

Appendix A — Technology Stack Summary

Technology / Library	Version & Purpose
HTML5 + CSS3	Single-file SPA structure, RTL Arabic layout
Vanilla JavaScript (ES6+)	Rule engine, CSV logic, UI interactions
Google Fonts — Tajawal	Arabic-optimized typeface for UI and PDF export
Chart.js	v4.4 — Doughnut and bar chart rendering
PapaParse	v5.4 — Client-side CSV parsing with header detection
Anthropic Claude API	claude-sonnet-4 — AI audit narrative generation
GitHub Pages	Static hosting of the live prototype

Table A1. Complete technology stack used in the Meezan prototype.